

Certainly. Below is a **master comparison chart for the major public concrete/general-purpose classes in the Java Collections Framework**, using the comparison parameters from the previous answer.

A literal “all parameters × all classes” table would become too wide to read, so I’ve divided the same comparison into logical tables: **architecture, behavior, ordering/null/duplicates, performance, concurrency, iteration, memory, and use cases.**

Scope note: This covers the standard public collection/map implementations developers normally compare from java.util and java.util.concurrent. Specialized utility/wrapper implementations returned by factory methods such as List.of() and internal JDK implementation classes are discussed separately.

1. Master list of concrete collection classes

Category	Concrete Class	Main Interface	Internal Structure	Primary Purpose
List	ArrayList	List	Resizable array	General-purpose indexed list
List	LinkedList	List, Deque	Doubly linked list	List + queue/deque
List	Vector	List	Synchronized resizable array	Legacy synchronized list
List	Stack	List via Vector	Resizable array	Legacy LIFO stack
Set	HashSet	Set	Hash table	Fast unique elements
Set	LinkedHashSet	Set	Hash table + linked ordering	Unique insertion-ordered elements
Set	TreeSet	NavigableSet	Red-black tree	Sorted unique elements
Queue	PriorityQueue	Queue	Binary heap	Priority-based processing
Deque	ArrayDeque	Deque	Resizable circular array	Fast queue/stack
Map	HashMap	Map	Hash table	General-purpose key/value lookup

Map	LinkedHashMap	Map	Hash table + linked ordering	Ordered map / LRU-style structures
Map	TreeMap	NavigableMap	Red-black tree	Sorted key/value mappings
Map	Hashtable	Map	Synchronized hash table	Legacy synchronized map
Map	WeakHashMap	Map	Hash table + weak keys	GC-sensitive mappings
Map	IdentityHashMap	Map	Identity-based hash table	Reference-identity mappings
Map	EnumMap	Map	Enum-indexed array-like structure	Maps whose keys are enum constants
Set	EnumSet*	Set	Bit-vector representation	Sets of enum constants
Concurrent List	CopyOnWriteArrayList	List	Copy-on-write array	Read-heavy concurrent list
Concurrent Set	CopyOnWriteArraySet	Set	Copy-on-write list internally	Read-heavy concurrent set
Concurrent Set	ConcurrentSkipListSet	NavigableSet	Skip list	Concurrent sorted set
Concurrent Queue	ConcurrentLinkedQueue	Queue	Linked non-blocking queue	Concurrent FIFO queue
Concurrent Deque	ConcurrentLinkedDeque	Deque	Linked non-blocking deque	Concurrent double-ended queue
Blocking Queue	ArrayBlockingQueue	BlockingQueue	Fixed array	Bounded blocking FIFO queue
Blocking Queue	LinkedBlockingQueue	BlockingQueue	Linked nodes	Optionally bounded blocking queue

Blocking Queue	PriorityBlockingQueue	BlockingQueue	Priority heap	Concurrent priority queue
Blocking Queue	DelayQueue	BlockingQueue	Priority-based delayed elements	Delay/expiration processing
Blocking Queue	SynchronousQueue	BlockingQueue	Direct handoff	Producer-consumer handoff
Blocking Queue	LinkedTransferQueue	TransferQueue	Linked concurrent nodes	Producer-consumer transfer
Blocking Deque	LinkedBlockingDeque	BlockingDeque	Linked nodes	Blocking double-ended queue
Concurrent Map	ConcurrentHashMap	ConcurrentMap	Concurrent hash table	Highly concurrent key/value map
Concurrent Map	ConcurrentSkipListMap	ConcurrentNavigableMap	Skip list	Concurrent sorted map

* EnumSet itself is abstract; instances are created through factory methods such as EnumSet.of(). It is included because it is an important standard set implementation family.

2. Fundamental behavior comparison

Class	Duplicates	Ordering	Sorted?	Index Access	Queue/Deque	Thread-Safe
ArrayList	✓	Insertion	✗	✓	✗	✗
LinkedList	✓	Insertion	✗	✓ but	✓	✗

				slow		
Vector	✓	Insertion	✗	✓	✗	✓ synchronized methods
Stack	✓	LIFO usage	✗	✓ inherited	Stack	✓ synchronized methods
HashSet	✗	Unspecified	✗	✗	✗	✗
LinkedHashSet	✗	Insertion	✗	✗	✗	✗
TreeSet	✗	Sorted	✓	✗	✗	✗
PriorityQueue	✓	Priority	Heap-ordered	✗	Queue	✗
ArrayDeque	✓	End/order based	✗	✗	✓	✗
CopyOnWriteArrayList	✓	Insertion	✗	✓	✗	✓
CopyOnWriteArraySet	✗	Implementation- preserved iteration order	✗	✗	✗	✓
ConcurrentSkipListSet	✗	Sorted	✓	✗	✗	✓
ConcurrentLinkedQueue	✓	FIFO	✗	✗	Queue	✓
ConcurrentLinkedDeque	✓	Deque order	✗	✗	Deque	✓

3. Map behavior comparison

Class	Duplicate Keys	Duplicate Values	Key Ordering	Sorted Keys	Thread-Safe
HashMap	✗	✓	Unspecified	✗	✗
LinkedHashMap	✗	✓	Insertion/access	✗	✗
TreeMap	✗	✓	Sorted	✓	✗
Hashtable	✗	✓	Unspecified	✗	✓
WeakHashMap	✗	✓	Unspecified	✗	✗
IdentityHashMap	Identity-based	✓	Unspecified	✗	✗
EnumMap	✗	✓	Enum declaration order	Natural enum order	✗
ConcurrentHashMap	✗	✓	Unspecified	✗	✓
ConcurrentSkipListMap	✗	✓	Sorted	✓	✓

4. Null-handling comparison

Null behavior is one of the most important parameters when choosing an implementation.

Class	Null Elements	Null Key	Null Values
ArrayList	✓ Multiple	N/A	N/A
LinkedList	✓ Multiple	N/A	N/A
Vector	✓ Multiple	N/A	N/A

Stack	✓	N/A	N/A
HashSet	✓ One	N/A	N/A
LinkedHashSet	✓ One	N/A	N/A
TreeSet	Generally ✗for natural ordering	N/A	N/A
PriorityQueue	✗	N/A	N/A
ArrayDeque	✗	N/A	N/A
HashMap	N/A	✓ One	✓
LinkedHashMap	N/A	✓ One	✓
TreeMap	N/A	Generally ✗	✓
Hashtable	N/A	✗	✗
WeakHashMap	N/A	✓	✓
IdentityHashMap	N/A	✓	✓
EnumMap	N/A	✗	✓
CopyOnWriteArrayList	✓	N/A	N/A
CopyOnWriteArraySet	✓ One	N/A	N/A
ConcurrentHashMap	N/A	✗	✗
ConcurrentSkipListMap	N/A	✗	✗
ConcurrentSkipListSet	✗	N/A	N/A

ConcurrentLinkedQueue	✗	N/A	N/A
ConcurrentLinkedDeque	✗	N/A	N/A

Why concurrent collections often reject null

Consider:

V value = map.get(key);

If a concurrent map allowed null values, null could ambiguously mean:

- the key doesn't exist, or
- the key exists and maps to null.

ConcurrentHashMap avoids that ambiguity by prohibiting null keys and values.

5. Underlying data structure comparison

Class	Internal Concept	Important Consequence
ArrayList	Dynamic array	Excellent random access
LinkedList	Doubly linked nodes	Cheap end insertion/removal but poor random access
Vector	Dynamic array	Similar to ArrayList with synchronization overhead
Stack	Vector	Legacy stack implementation

HashSet	Hash table	Fast membership testing
LinkedHashSet	Hash table + linked ordering	Fast lookup + predictable iteration
TreeSet	Red-black tree	$O(\log n)$ sorted operations
PriorityQueue	Binary heap	Efficient priority retrieval
ArrayDeque	Resizable circular array	Efficient stack/queue operations
HashMap	Hash table/buckets	Fast average lookup
LinkedHashMap	Hash table + linked ordering	Ordered map traversal
TreeMap	Red-black tree	Sorted keys
WeakHashMap	Weak-key hash table	Entries can disappear after GC
IdentityHashMap	Identity-based hash table	Uses reference identity
EnumMap	Enum-oriented array representation	Extremely efficient enum keys
EnumSet	Bit vector	Extremely compact enum set
ConcurrentHashMap	Concurrent hash structure	Scalable concurrent access
ConcurrentSkipListMap	Skip list	Concurrent sorted map
ConcurrentSkipListSet	Skip list	Concurrent sorted set
CopyOnWriteArrayList	Copy-on-write array	Reads cheap; writes expensive

6. Time-complexity master chart

n = number of elements.

Class	Get/Lookup	Add/Put	Remove	Contains/Search
ArrayList	O(1) index	O(1)* append	O(n)	O(n)
LinkedList	O(n)	O(1) ends	O(n) search	O(n)
Vector	O(1)	O(1)*	O(n)	O(n)
Stack	O(1) top	O(1)* push	O(1) pop	O(n)
HashSet	N/A	O(1) avg	O(1) avg	O(1) avg
LinkedHashSet	N/A	O(1) avg	O(1) avg	O(1) avg
TreeSet	N/A	O(log n)	O(log n)	O(log n)
PriorityQueue	O(1) peek	O(log n)	O(log n)**	O(n) arbitrary search
ArrayDeque	O(1) ends	O(1)* ends	O(1) ends	O(n)
HashMap	O(1) avg	O(1) avg	O(1) avg	O(1) avg key
LinkedHashMap	O(1) avg	O(1) avg	O(1) avg	O(1) avg key
TreeMap	O(log n)	O(log n)	O(log n)	O(log n)
EnumMap	O(1)	O(1)	O(1)	O(1)
ConcurrentHashMap	O(1) avg	O(1) avg	O(1) avg	O(1) avg
ConcurrentSkipListMap	O(log n)	O(log n)	O(log n)	O(log n)

ConcurrentSkipListSet	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
CopyOnWriteArrayList	$O(1)$	$O(n)$	$O(n)$	$O(n)$

* Amortized.

** Removing the head is $O(\log n)$; locating and removing an arbitrary object may require linear search.

7. Random vs sequential access

Class	Random Access	Sequential Access	Best Access Pattern
ArrayList	★ ★ ★ ★ ★	★ ★ ★ ★ ★	Index-based reads
LinkedList	★	★ ★ ★ ★	Ends/iterator traversal
Vector	★ ★ ★ ★ ★	★ ★ ★ ★	Legacy synchronized indexed access
CopyOnWriteArrayList	★ ★ ★ ★ ★	★ ★ ★ ★ ★	Concurrent read-heavy access
HashSet	N/A	Iteration	Membership
LinkedHashSet	N/A	Excellent predictable iteration	Ordered uniqueness
TreeSet	N/A	Sorted traversal	Range/sorted access
ArrayDeque	N/A	Excellent	Head/tail access
PriorityQueue	N/A	Not sorted traversal	Priority head

For example:

```
List<String> list = new ArrayList<>();
```

```
list.add("A");  
list.add("B");  
list.add("C");
```

```
System.out.println(list.get(2));
```

Output:

C

ArrayList.get(2) is O(1).

8. Duplicate handling

List

```
List<String> list = new ArrayList<>();
```

```
list.add("Java");  
list.add("Java");  
list.add("Spring");
```

```
System.out.println(list);
```

Output:

[Java, Java, Spring]

Lists allow duplicates.

Set

```
Set<String> set = new HashSet<>();
```

```
set.add("Java");  
set.add("Java");  
set.add("Spring");
```

```
System.out.println(set.size());
```

Output:

2

Sets enforce uniqueness.

9. Equality mechanism comparison

Class	Equality Mechanism
ArrayList	Element equals() + order
LinkedList	Element equals() + order
HashSet	hashCode() + equals()
LinkedHashSet	hashCode() + equals()
TreeSet	Ordering comparison
HashMap	Keys: hashCode() + equals()
LinkedHashMap	Keys: hashCode() + equals()

TreeMap	Key comparison
IdentityHashMap	== reference identity
EnumMap	Enum keys
ConcurrentHashMap	Hash/equality semantics

The exceptional case is IdentityHashMap.

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a.equals(b));
System.out.println(a == b);
```

Output:

```
true
false
```

A normal HashMap treats these keys as equal.

An IdentityHashMap treats them as different because it uses reference identity rather than ordinary object equality.

10. Natural ordering and Comparator support

Class	Comparable	Comparator	Automatically Sorted
ArrayList	Not required	Sorting can use one	✗
LinkedList	Not required	Sorting can use one	✗

HashSet	Not required	✗ inherent sorting	✗
LinkedHashSet	Not required	✗ inherent sorting	✗
TreeSet	✓ Natural ordering option	✓	✓
PriorityQueue	✓ Natural ordering option	✓	Priority
TreeMap	Keys may use natural ordering	✓	✓
ConcurrentSkipListSet	Natural ordering option	✓	✓
ConcurrentSkipListMap	Natural ordering option	✓	✓

Example:

```
Set<Integer> numbers =
    new TreeSet<>(Comparator.reverseOrder());
```

```
numbers.add(10);
numbers.add(30);
numbers.add(20);
```

```
System.out.println(numbers);
```

Output:

```
[30, 20, 10]
```

11. Capacity and resizing

Class	Capacity Concept	Resizable	Capacity Configuration
ArrayList	Backing array	✓	Initial capacity
LinkedList	Nodes	Naturally grows	N/A
Vector	Backing array	✓	Initial capacity/capacity increment
HashSet	Hash table	✓	Capacity + load factor
LinkedHashSet	Hash table	✓	Capacity + load factor
HashMap	Hash table	✓	Capacity + load factor
LinkedHashMap	Hash table	✓	Capacity + load factor
PriorityQueue	Heap array	✓	Initial capacity
ArrayDeque	Internal array	✓	Automatically
ArrayBlockingQueue	Array	✗ fixed	Constructor capacity
LinkedBlockingQueue	Nodes	Bounded/large capacity	Constructor
ConcurrentHashMap	Hash table	✓	Initial sizing options

Example:

```
List<Integer> list = new ArrayList<>(1000);
```

This can be useful when approximately 1,000 elements are expected, reducing repeated resizing.

12. Load-factor comparison

Load factor primarily applies to hash-table-based implementations.

Class	Hash Based	Load Factor Relevant?
HashMap	✓	✓
HashSet	✓	✓
LinkedHashMap	✓	✓
LinkedHashSet	✓	✓
Hashtable	✓	✓
WeakHashMap	✓	Implementation concern
ConcurrentHashMap	✓	Sizing behavior differs from ordinary HashMap
TreeMap	✗	✗
TreeSet	✗	✗
ArrayList	✗	✗
LinkedList	✗	✗

For ordinary HashMap, the familiar default load factor is:

0.75

Example:

```
Map<Integer, String> map =  
    new HashMap<>(100, 0.75f);
```

13. Memory-overhead comparison

Exact memory consumption depends on the JVM, compressed references, object layout and capacity, so relative comparisons are more useful.

Class	Relative Overhead	Main Reason
ArrayList	Low	Array of references
LinkedList	High	Node + previous/next references
HashSet	Medium/High	Hash table/map entries
LinkedHashSet	High	Hashing + linked ordering
TreeSet	High	Tree nodes
ArrayDeque	Low	Array
PriorityQueue	Low/Medium	Heap array
HashMap	Medium	Bucket/table entries
LinkedHashMap	Higher	Hash entries + ordering links

TreeMap	High	Tree nodes
EnumSet	Very low	Bit-vector representation
EnumMap	Very low	Specialized enum representation
CopyOnWriteArrayList	Potentially high during writes	Copies array
ConcurrentHashMap	Higher than basic map	Concurrency infrastructure

14. Iterator behavior

Collection	Typical Iterator Behavior	Concurrent Modification
ArrayList	Fail-fast	Usually ConcurrentModificationException
LinkedList	Fail-fast	Usually exception
HashSet	Fail-fast	Usually exception
LinkedHashSet	Fail-fast	Usually exception
TreeSet	Fail-fast	Usually exception
HashMap views	Fail-fast	Usually exception
TreeMap views	Fail-fast	Usually exception
CopyOnWriteArrayList	Snapshot	Does not reflect later modifications
ConcurrentHashMap	Weakly consistent	Concurrent modification allowed
ConcurrentLinkedQueue	Weakly consistent	Concurrent modification allowed

ConcurrentSkipListMap	Weakly consistent	Concurrent modification allowed
------------------------------	-------------------	---------------------------------

“Fail-fast” is best understood as a **bug-detection behavior**, not a thread-safety guarantee.

15. Thread-safety master chart

Class	Thread-Safe	Mechanism/Approach	Best Concurrent Replacement
ArrayList	✗	None	CopyOnWriteArrayList depending on workload
LinkedList	✗	None	Concurrent queue/deque depending on requirement
HashSet	✗	None	ConcurrentHashMap.newKeySet()
TreeSet	✗	None	ConcurrentSkipListSet
HashMap	✗	None	ConcurrentHashMap
TreeMap	✗	None	ConcurrentSkipListMap
Vector	✓	Method synchronization	Often ArrayList + appropriate synchronization/concurrent design
Stack	✓ methods inherited from Vector	Synchronization	Usually ArrayDeque for non-concurrent stacks
Hashtable	✓	Synchronization	ConcurrentHashMap
CopyOnWriteArrayList	✓	Copy-on-write	—

CopyOnWriteArraySet	✓	Copy-on-write	—
ConcurrentHashMap	✓	Concurrent algorithms	—
ConcurrentSkipListMap	✓	Concurrent skip list	—
ConcurrentSkipListSet	✓	Concurrent skip list	—

16. Blocking-queue comparison

These classes are especially important in multithreading and executor-based systems.

Class	Bounded?	Ordering	Blocking	Typical Application
ArrayBlockingQueue	✓	FIFO	✓	Fixed-size producer/consumer
LinkedBlockingQueue	Optional	FIFO	✓	General producer/consumer
PriorityBlockingQueue	Effectively unbounded	Priority	✓ retrieval	Priority task processing
DelayQueue	Unbounded	Expiration/delay	✓	Expiring/delayed tasks
SynchronousQueue	Zero capacity	Handoff	✓	Direct producer→consumer
LinkedTransferQueue	Effectively unbounded	FIFO-style	Supports transfer/waiting	High-throughput producer/consumer
LinkedBlockingDeque	Optional	Double-ended	✓	Blocking deque

Example:

```
BlockingQueue<String> queue =
```

```
new ArrayBlockingQueue<>(100);
```

```
queue.put("Task-1");
```

```
String task = queue.take();
```

put() can wait for space, while take() can wait for an element.

This makes blocking queues extremely useful in **producer-consumer architectures**.

17. CopyOnWriteArrayList vs ArrayList

Parameter	ArrayList	CopyOnWriteArrayList
Thread-safe	✗	✓✓
Random access	O(1)	O(1)
Read performance	Excellent	Excellent
Write performance	Good	Expensive
Write operation	Modify backing array	Copy backing array
Iterator	Fail-fast	Snapshot
Concurrent iteration	Unsafe without coordination	Safe
Best workload	Normal single-thread/general use	Many reads + very few writes

For example, it is useful for listener lists:

```
CopyOnWriteArrayList<EventListener> listeners =  
    new CopyOnWriteArrayList<>();
```

If listeners are read thousands of times but added/removed rarely, copy-on-write behavior can be a good fit.

18. Specialized collection comparison

Class	Special Feature	Use When
EnumSet	Extremely efficient enum set	Elements are enum constants
EnumMap	Extremely efficient enum-key map	Keys are one enum type
WeakHashMap	Weakly referenced keys	Entries should be removable after keys become otherwise unreachable
IdentityHashMap	== identity semantics	Object identity matters
PriorityQueue	Priority-based head	Need priority processing
DelayQueue	Delayed availability	Scheduling/expiration
SynchronousQueue	No storage	Direct thread handoff

EnumMap example

```
enum Status {  
    NEW,  
    PROCESSING,  
    COMPLETED
```

```

}

Map<Status, String> messages =
    new EnumMap<>(Status.class);

messages.put(Status.NEW, "Order created");
messages.put(Status.PROCESSING, "Order processing");
messages.put(Status.COMPLETED, "Order completed");

```

When the keys are enum constants, EnumMap should usually be considered before HashMap.

19. Mutability comparison

Type	Example	Add/Remove Allowed?
Mutable	new ArrayList<>()	✓
Mutable	new HashSet<>()	✓
Mutable	new HashMap<>()	✓
Fixed-size list	Arrays.asList(...)	Structural add/remove ✗
Unmodifiable factory collection	List.of(...)	✗
Unmodifiable factory set	Set.of(...)	✗
Unmodifiable factory map	Map.of(...)	✗
Unmodifiable view	Collections.unmodifiableList(...)	✗ through view

These concepts are different.

```
List<String> original = new ArrayList<>();  
original.add("Java");
```

```
List<String> view =  
    Collections.unmodifiableList(original);
```

```
original.add("Spring");
```

```
System.out.println(view);
```

The view reflects changes to the underlying list even though modification through the view is prohibited.

20. Stream and functional-programming support

Classes implementing Collection inherit operations such as:

```
stream()  
parallelStream()  
removeIf()  
forEach()  
spliterator()
```

Example:

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);
```

```
List<Integer> result =  
    numbers.stream()  
        .filter(n -> n >= 20)  
        .map(n -> n * 2)
```

```
.toList();
```

```
System.out.println(result);
```

Output:

```
[40, 50, 60]
```

Map is different because it does not extend Collection.

Typically:

```
map.entrySet().stream()
```

```
map.keySet().stream()
```

```
map.values().stream()
```

21. Best class according to requirement

Requirement	Usually Consider First
General-purpose list	ArrayList
Fast index access	ArrayList
Frequent deque operations	ArrayDeque
Stack	ArrayDeque
FIFO queue	ArrayDeque
Unique elements	HashSet
Unique + insertion order	LinkedHashSet

Unique + sorted	TreeSet
Priority processing	PriorityQueue
General key/value map	HashMap
Map + predictable insertion order	LinkedHashMap
Sorted map	TreeMap
Enum set	EnumSet
Enum-key map	EnumMap
Concurrent map	ConcurrentHashMap
Concurrent sorted map	ConcurrentSkipListMap
Concurrent sorted set	ConcurrentSkipListSet
Concurrent read-heavy list	CopyOnWriteArrayList
Concurrent FIFO	ConcurrentLinkedQueue
Concurrent deque	ConcurrentLinkedDeque
Bounded producer-consumer queue	ArrayBlockingQueue
Blocking FIFO	LinkedBlockingQueue
Priority + blocking	PriorityBlockingQueue
Delayed processing	DelayQueue
Direct producer-consumer handoff	SynchronousQueue

22. Overall decision chart

Requirement	Array List	Linked List	Hash Set	Linked HashSet	Tree Set	Array Deque	HashMap	LinkedHash Map	Tree Map	Concurrent HashMap
Duplicates	✓	✓	✗	✗	✗	✓	Values ✓	Values ✓	Values ✓	Values ✓
Insertion order	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗
Sorted	✗	✗	✗	✗	✓	✗	✗	✗	✓	✗
Index access	☆☆☆ ☆☆	☆	—	—	—	—	—	—	—	—
Search	☆	☆	☆☆☆ ☆☆	☆☆☆☆☆	☆☆☆ ☆	☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆ ☆	☆☆☆☆☆
Add	☆☆☆ ☆☆	☆☆☆☆☆ ends	☆☆☆ ☆☆	☆☆☆☆☆	☆☆☆ ☆	☆☆☆ ☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆ ☆	☆☆☆☆☆
Memory efficiency	☆☆☆ ☆	☆☆	☆☆☆	☆☆	☆☆	☆☆☆ ☆☆	☆☆☆	☆☆	☆☆	☆☆☆
Thread safety	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Sorted range operations	✗	✗	✗	✗	☆☆☆ ☆☆	✗	✗	✗	☆☆☆ ☆☆	✗

The stars are **relative conceptual ratings**, not benchmark measurements.

23. The most important selection rule

The Java collection-selection problem can be reduced to a decision sequence:

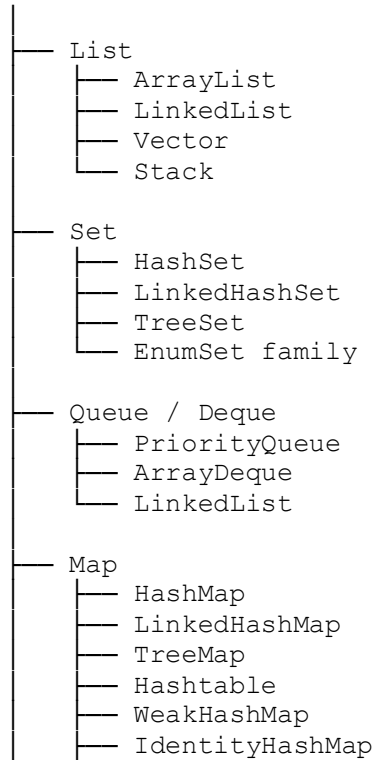
Question	If Yes	Usually Choose
Need key/value pairs?	Yes	Map family
Need unique elements?	Yes	Set family
Need duplicates + positional/index access?	Yes	List family
Need FIFO/LIFO processing?	Yes	Queue / Deque
Need automatic sorting?	Yes	TreeSet / TreeMap
Need insertion order + uniqueness?	Yes	LinkedHashSet
Need insertion order for map?	Yes	LinkedHashMap
Need fastest ordinary unique lookup?	Yes	HashSet
Need fastest ordinary key lookup?	Yes	HashMap
Need fast indexed access?	Yes	ArrayList
Need stack/deque operations?	Yes	ArrayDeque
Need priority processing?	Yes	PriorityQueue
Need thread-safe hash map?	Yes	ConcurrentHashMap
Need concurrent sorted map?	Yes	ConcurrentSkipListMap

Need read-heavy concurrent list?	Yes	CopyOnWriteArrayList
Need blocking producer-consumer behavior?	Yes	BlockingQueue implementation
Keys are enums?	Yes	EnumMap
Unique values are enums?	Yes	EnumSet

Final mental model

Think of the framework as:

Java Collections Framework





The core selection principle is:

ArrayList	→ indexed ordered data
HashSet	→ unique fast lookup
LinkedHashSet	→ unique + insertion order
TreeSet	→ unique + sorted
ArrayDeque	→ stack/queue/deque
PriorityQueue	→ priority processing
HashMap	→ fast key/value lookup
LinkedHashMap	→ ordered key/value data
TreeMap	→ sorted key/value data
ConcurrentHashMap	→ concurrent key/value access

This comparison framework—**interface** → **data structure** → **ordering** → **duplicates** → **null** → **equality** → **access** → **insertion** → **deletion** → **search** → **memory** → **iteration** → **synchronization** → **concurrency** → **mutability** → **streams** → **use case**—can be used to compare virtually any two Java collection implementations.